

HDOC Day-30

NAME: Yug Nirwal
SAP ID: 590033354
SUBJECT: C programming
DATE: 23-09-26

Question

Write a C program to perform the following menu-driven operations on a given array: (1) sort the elements using Bubble Sort and perform a Binary Search, (2) perform insertion and deletion at a valid position, (3) rotate the array elements to the right by k positions, and (4) exit.

Algorithm

1. Read the size and elements of the array.
2. Display the menu and read the user's choice.
3. For choice 1, sort the array using Bubble Sort. Then read the search value and use Binary Search on the sorted array.
4. For choice 2, ask whether to insert or delete. For insertion, shift elements right and place the new value. For deletion, shift elements left from the selected position.
5. For choice 3, read k, reduce it using $k \% n$, and rotate the array to the right by k positions using a temporary array.
6. Repeat the menu until the user chooses Exit.

Test Cases

Test	Operation	Input / Condition	Expected Result
1	Bubble Sort + Binary Search	5 2 8 1 4; search 4	Sorted: 1 2 4 5 8; found at position 3
2	Bubble Sort + Binary Search	9 3 7 1; search 6	Sorted: 1 3 7 9; not found
3	Insertion	10 20 30 40 50; insert 25 at pos 3	10 20 25 30 40 50
4	Deletion	10 20 30 40 50; delete pos 4	10 20 30 50
5	Right Rotation	1 2 3 4 5 6; k = 2	5 6 1 2 3 4

C Program

```
#include <stdio.h>

void display(int a[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
            }
}

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int a[100], n, choice;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    do {
        printf("\n1. Bubble Sort and Binary Search\n");
        printf("2. Insertion and Deletion\n");
        printf("3. Rotate array to right\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            int key, pos;
            bubbleSort(a, n);
            printf("Sorted array: "); display(a, n);
            printf("Enter element to search: "); scanf("%d", &key);
            pos = binarySearch(a, n, key);
            if (pos == -1) printf("Element not found\n");
            else printf("Element found at position %d\n", pos + 1);
        }
        else if (choice == 2) {
            int op, pos, value;
            printf("1. Insert\n2. Delete\nEnter operation: "); scanf("%d", &op);
            if (op == 1) {
                printf("Enter position: "); scanf("%d", &pos);
                if (pos < 1 || pos > n + 1 || n >= 100) printf("Invalid position\n");
                else {
                    printf("Enter value: "); scanf("%d", &value);
                    for (int i = n; i >= pos; i--) a[i] = a[i - 1];
                    a[pos - 1] = value; n++;
                    printf("Array after insertion: "); display(a, n);
                }
            }
            else if (op == 2) {
                printf("Enter position: "); scanf("%d", &pos);
                if (pos < 1 || pos > n) printf("Invalid position\n");
                else {
                    for (int i = pos - 1; i < n - 1; i++) a[i] = a[i + 1];
                    n--; printf("Array after deletion: "); display(a, n);
                }
            }
        }
        else if (choice == 3) {
            int k, temp[100];
            if (n == 0) { printf("Array is empty\n"); continue; }
            printf("Enter k: "); scanf("%d", &k); k %= n;
            for (int i = 0; i < n; i++) temp[(i + k) % n] = a[i];
            for (int i = 0; i < n; i++) a[i] = temp[i];
            printf("Array after right rotation: "); display(a, n);
        }
    } while (choice != 4);
}
```

```
}    return 0;
```

Execution / Test Case Outputs

Test Case 1

```
(kali~kali)-[~/Desktop/cprog]
$ ./a.out
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to right
4. Exit
Enter your choice: 1
Enter number of elements: 5
Enter elements: 5 2 8 1 4
Sorted array: 1 2 4 5 8
Enter element to search: 4
Element found at position 3
```

Test Case 2

```
(kali~kali)-[~/Desktop/cprog]
$ ./a.out
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to right
4. Exit
Enter your choice: 1
Enter number of elements: 4
Enter elements: 9 3 7 1
Sorted array: 1 3 7 9
Enter element to search: 6
Element not found
```

Test Case 3

```
(kali~kali)-[~/Desktop/cprog]
$ ./a.out
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to right
4. Exit
Enter your choice: 2
Enter number of elements: 5
Enter elements: 10 20 30 40 50
1. Insert
2. Delete
Enter operation: 1
Enter position: 3
Enter value: 25
Array after insertion: 10 20 25 30 40 50
```

Test Case 4

```
(kali~kali)-[~/Desktop/cprog]
$ ./a.out
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to right
4. Exit
Enter your choice: 2
Enter number of elements: 5
Enter elements: 10 20 30 40 50
1. Insert
2. Delete
Enter operation: 2
Enter position: 4
Array after deletion: 10 20 30 50
```

Test Case 5

```
(kali~kali)-[~/Desktop/cprog]
$ ./a.out
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to right
4. Exit
Enter your choice: 3
Enter number of elements: 6
Enter elements: 1 2 3 4 5 6
Enter k: 2
Array after right rotation: 5 6 1 2 3 4
```

Result

The menu-driven C program was written and tested successfully. Bubble sorting with binary search, insertion, deletion, and right rotation produced the expected outputs for the prepared test cases.